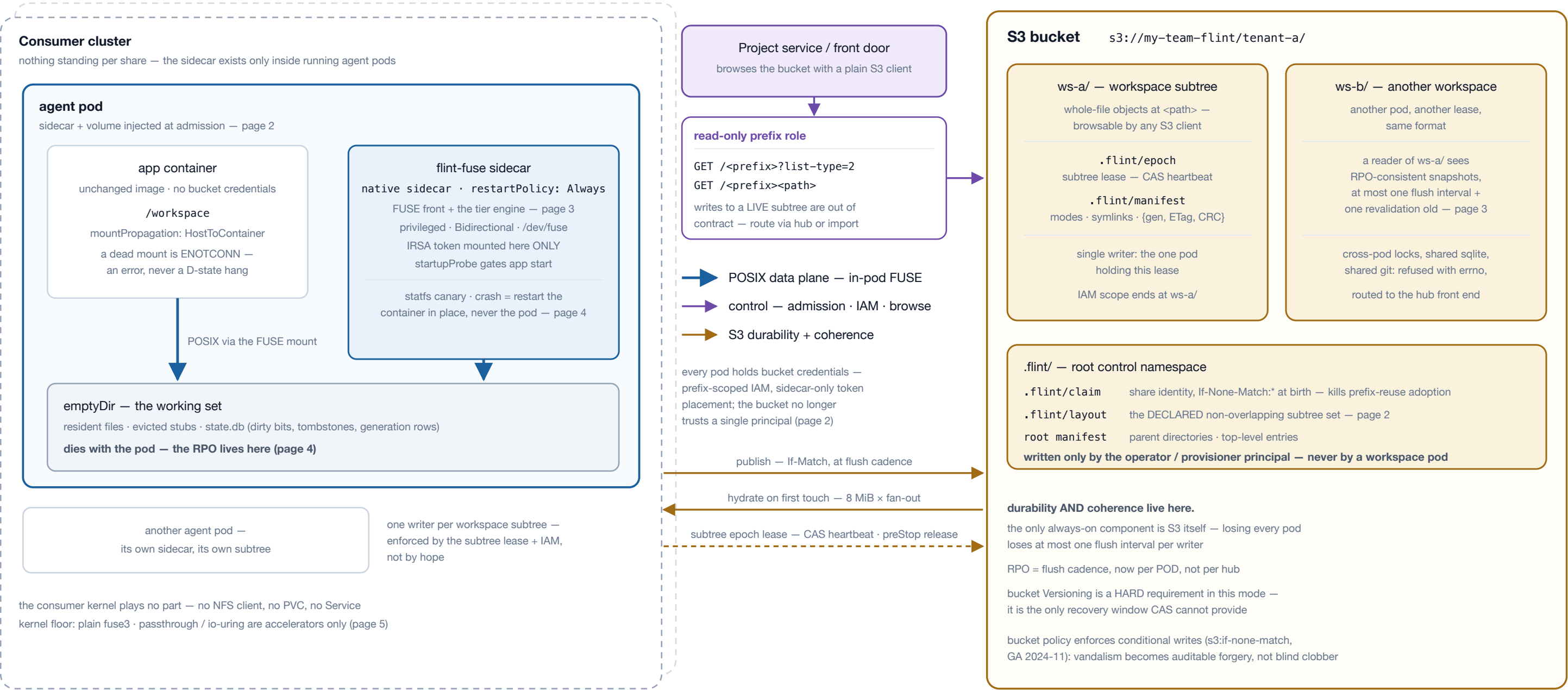
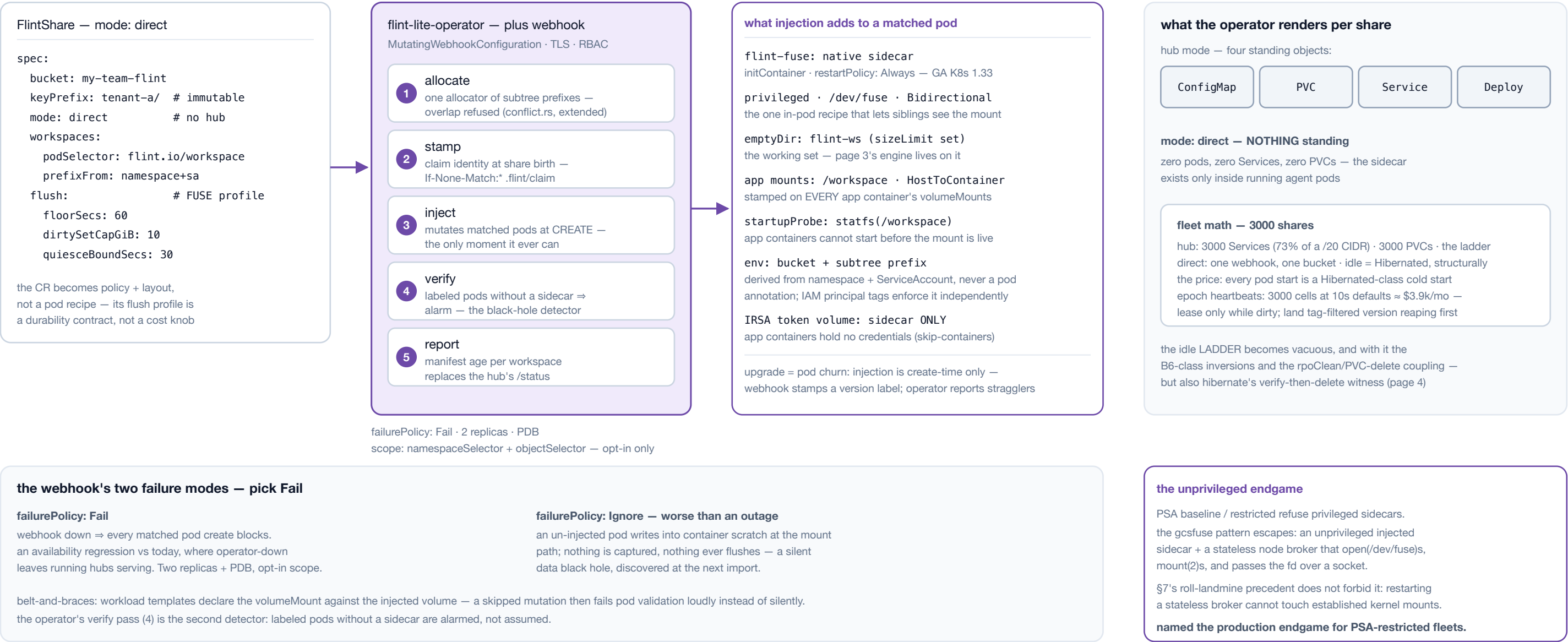


The proposal: no hub. A webhook-injected sidecar runs the same tier engine behind a FUSE mount; agents and the project service reach S3 directly; the bucket format — whole-file objects, `.flint/` control cells, guarded publishes — is unchanged, so hub-mode and direct-mode volumes stay mutually convertible. Reviewed adversarially against the code on 2026-08-24 (14 agents, 7 dimensions); the corrections are folded into these pages. A lighter direct-mode variant that needs no FUSE at all — checkout/publish plus a write gateway — has its own doc: `flint-lean-architecture`.

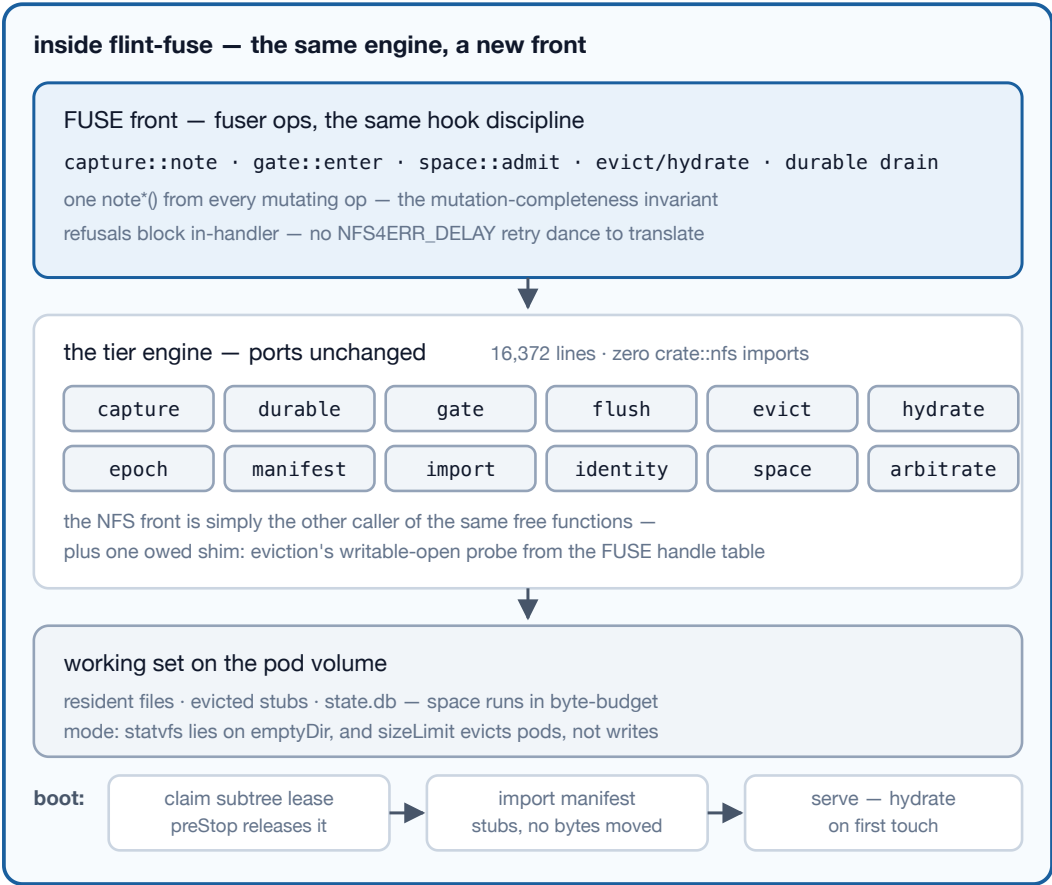


The hub column is gone, and with it the coherence authority. What replaces it is the store's own CAS — If-Match on every publish, a lease cell per workspace — and CAS is detection and fencing, not serialization: it binds only writers that cooperate. The contract therefore narrows to **one writer per workspace subtree**, and the honest name for what a reader of someone else's subtree sees is **per-subtree RPO-consistent snapshots**: a consistent image at most one flush interval (~60s floor) plus one revalidation old — the same number that bounds crash loss, so there is one lag to explain, not two. Two things are true at once and both belong in the pitch: there is no share-wide single point of failure left, and per-pod durability is strictly worse than the hub's PVC, because the working set now dies with the pod (page 4). **The project service browses read-only, deliberately.** A console PUT against a live subtree has three fates, none acceptable: silently destroyed by the writer's local-wins re-publish, undetected until eviction, or — worst — adopted as truth by a later hydration 412 and served to the agent. Write through the hub front end or an explicit import flow. **The bucket no longer trusts one principal.** Every pod holds prefix-scoped credentials (IRSA, token mounted in the sidecar container only); compared honestly against today's AUTH_SYS-on-:2049 with NetworkPolicy off by default, that narrows cross-tenant reach — while irreducibly widening what a compromised pod can do to its own subtree, `.flint/` cells included, because flint-fuse itself is the epoch holder and manifest publisher.

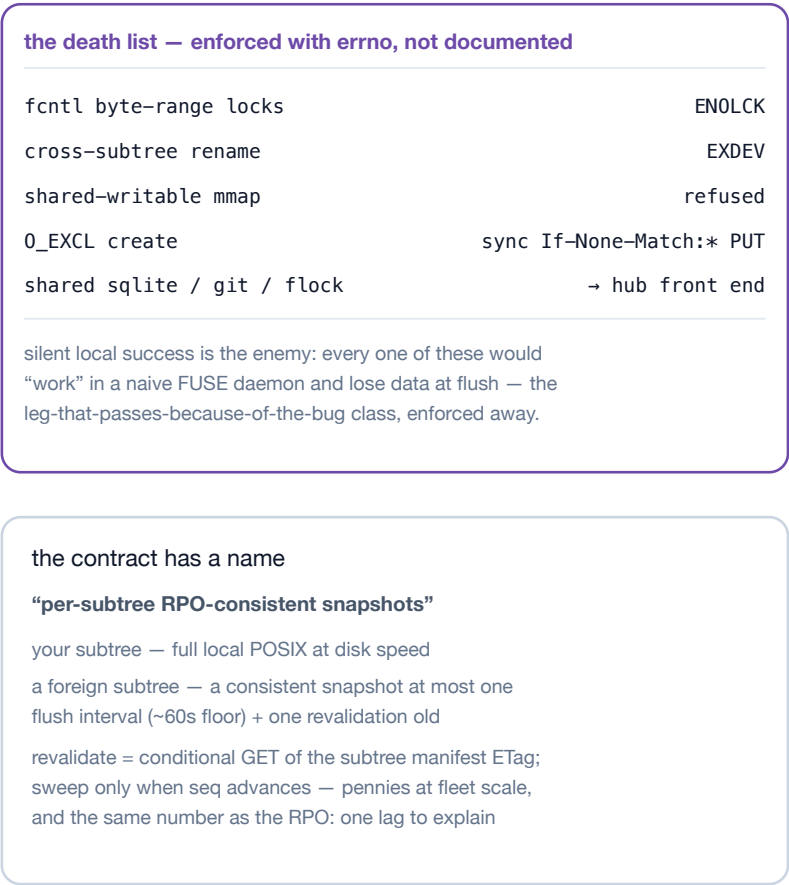
One CR still governs a share, but in direct mode it renders no standing objects — the operator's job becomes admission: allocate non-overlapping subtree prefixes, stamp the claim identity, inject the sidecar into matched pods, and detect the pods that slipped past.



src/tier — capture, durable, gate, flush, evict, hydrate, epoch, manifest, import — ports into the sidecar unchanged; the FUSE front reproduces the NFS front's hook discipline. What does not port is the coherence inversion: reading other writers' work while serving, under leases scoped to subtrees.



boot lists nothing: manifest GET first, jittered — a 1000-pod reclaim wave must not LIST an unpartitioned bucket into a 503 storm



The honest headline is not “no single point of failure.” Hub mode concentrates failure into rare, share-wide, hang-shaped outages; direct mode distributes it into frequent, per-pod, loss-shaped events. The full matrix — including the two rows the proposal creates and the two it deletes.

not “no single point of failure” — a failure-shape trade

hub mode concentrates failure into rare, share-wide, HANG-shaped outages; FUSE mode distributes it into frequent, per-pod, LOSS-shaped events.
durability becomes an operational statistic — interruption handlers, grace periods, dirty-set caps — instead of an architecture property.

event	hub mode — today (measured)	FUSE mode — proposed
hub pod crash	restart + epoch re-claim $\approx$ 13s; clients park and resume — nothing durable at risk (PVC)	row deleted — there is no hub
hub NODE crash	every client hard-hangs in D-state; Recreate + RWO + ~6 min force-detach; F29's manual arm	row deleted
FUSE daemon crash	n/a	ENOTCONN — an error apps can see; restart-in-place recovers the emptyDir dirty set; open fds stay severed — apps reopen
agent pod spot reclaim	mount unaffected — pods reschedule	preStop drain inside the 120s notice — IF the interruption handler + grace wiring exists; dirty beyond the ~9–15 GiB bound is lost
agent NODE crash	mount unaffected	drains nothing — loses the unflushed set (hot files: unbounded, see RPO arithmetic) + a 60–110s subtree write-lockout
S3 outage / throttle storm	RPO grows; the PVC rides it out; one coordinated retry queue	N uncoordinated engines, N$\times$ re-fetch of the same objects; outage + reclaim COMPOUND $\Rightarrow$ that pod's dirty set is gone
webhook down	row does not exist	Fail: matched pod creates block · Ignore: the silent black hole — page 2
operator down	running hubs keep serving; wake / reconcile blocked	nothing standing to break; prefix allocation and stale-sidecar reporting pause

availability event — hangs or blocks, nothing durable at risk durability event — acked data at risk no impact / not applicable

the RPO arithmetic

floor 60s — the minimum loss on any unclean death
quiescence skips files written <10s apart — a WAL or agent log may NEVER have published: a pressure valve is required
preStop drain at the measured publish rate (8–13 s/GiB) $\approx$ 9–15 GiB inside the 120s spot notice — only with an interruption handler + terminationGracePeriodSeconds $\geq$ 120 (the K8s default 30s drains ~2–3 GiB) · hard crash drains 0

state the RPO per mode; measure the event rate on the fleet

recovery order is a contract

statsd canary — dead FUSE fails FAST; a liveness probe alone misses a hung-but-alive daemon
restart the sidecar container IN PLACE $\rightarrow$ reopen the dirty-bit db $\rightarrow$ whole-file-dirty recovery flush $\rightarrow$ only then recycle
the reflex — restart the pod — destroys exactly the dirty set a container restart would have saved

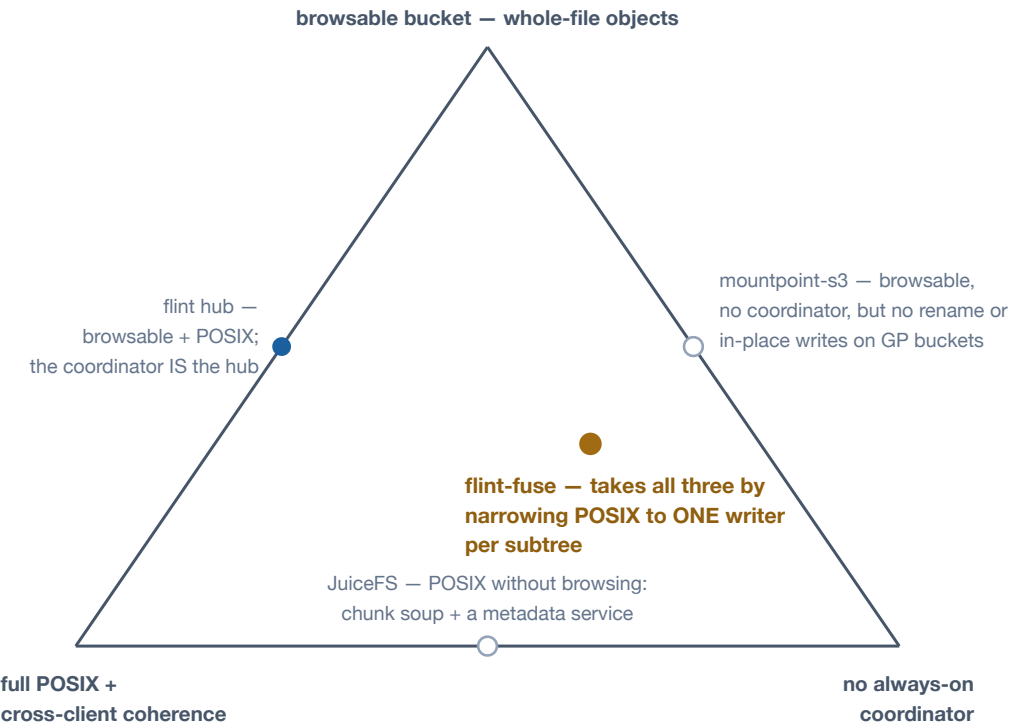
the lease tax on unclean death

a fresh emptyDir defeats self-recognition — the occupancy proof is gone: 60s configured, 80–110s measured, before the subtree is writable again
preStop clean release = the 17s fast path · do NOT retune the lease instead — 30s $\times$ 4 was tried and reverted

Where FUSE mode is strictly better: read-mostly and single-writer ephemeral workspaces whose dirty sets stay inside what the spot notice can drain (~9–15 GiB at the measured publish rate), whose owners accept losing up to one flush interval of their own recent work when a node dies — in exchange, no workspace ever hangs on another's failure, and a dead mount is an errno, not a D-state. **Where it is strictly worse:** shared-RWX trees, multi-pod sqlite/git, large-artifact writers, and any lifecycle that gates destructive actions on `rpoClean` — today's hibernate refuses to delete a PVC it cannot verify; FUSE mode has no equivalent witness, so the bucket (manifest seq and age per workspace, CAS-versioned and readable by a plain S3 client) becomes the only global status surface, and alerting moves there along with webhook-injected per-pod metrics. **Do the event-rate arithmetic with fleet numbers, not vibes:** on a pure-spot posture, pod terminations are thousands per month against roughly one hub exposure — most drain cleanly through preStop *if* an interruption handler (aws-node-termination-handler or Karpenter's) converts the 2-minute IMDS notice into prompt eviction, which Kubernetes does not do by itself, and the grace period is genuinely $\geq$ 120s. The residual — hard node deaths, oversized dirty sets, hot files the quiescence guard never flushed — is a permanent-loss class hub mode does not have; its rate is a property of the fleet and must be measured there, then stated on the CRD surface as the FUSE-mode RPO contract. The self-recognition fix is *not* a workspace-stable holder identity — the occupancy lock is precisely what proves the old incarnation is gone, and identity reuse would let a replacement depose a live mid-flush writer. Clean release on preStop, or an external fencing actor that has witnessed pod deletion finality.

When to route a workload to which front end, what must be measured and modeled before any code, and where the placement dial sits between per-pod isolation and per-node economics.

pick two — until you narrow “POSIX”



the triangle is forced by S3’s primitives — CAS detects and fences, it never serializes; flint-fuse's move is to shrink the contract until CAS is enough, and route everything larger to the hub.

four ways to serve a bucket

	hub	fuse	mp-s3	JuiceFS
browsable whole-file bucket	●	●	●	○
in-place writes + rename	●	●	○	●
cross-pod locks / shared writers	●	○	○	●
always-on component	the hub	S3 only	S3 only	metadata DB
idle cost	ladder → S3	S3, always	S3, always	DB + S3
acked-write crash durability	PVC	flush cadence	n/a	chunk store
consumer node needs	kernel NFS	fuse3 sidecar	FUSE	FUSE + DB

● yes ○ no

mountpoint-s3 is the read lane AWS keeps improving — re-run this survey at each of its minor releases; the moat is the working set + guarded publishes + manifest restore, where AWS has shown no motion

route to the hub

shared-RWX trees · multi-pod sqlite / git
large-artifact writers (dirty > ~10 GiB)
anything gating destructive ops on rpoClean
cross-subtree consistency points (EXDEV callers)
zero consumer requirements — kernel NFS only

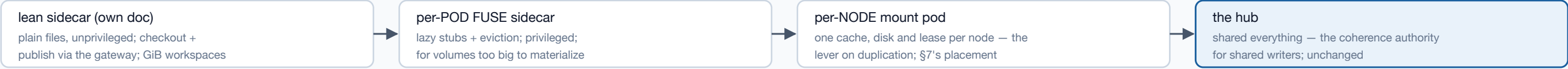
route to direct mode

one writer per workspace — the agent-fleet shape;
spot-heavy, idle-heavy, thousands of shares
LEAN (checkout/publish — its own doc) by default;
flint-fuse when the volume is too large or too
lazy to materialize · loss bound: ≤ one flush

gates before any code

- measure hydrate WITH the fan-out vs real S3 — 72.5 s/GiB measured pre-fan-out vs 1–13 theoretical; decides whether per-pod cold start is viable at all
- burst rig — N≥1000 correlated pod starts against an unpartitioned bucket (503 SlowDown during recovery)
- formal models — subtree lease + online refresh (“the abstraction was the bug”, three times now)
- page 3's four preconditions landed — claim identity, manifest sharding, removal journal, space budget

placement is a dial, not a doctrine — lightest that fits, escalate when it stops fitting



Buy fails twice, so build — on what already exists. Every performant S3-FUSE client refuses the write shapes agent workloads need (mountpoint-s3: no rename and no in-place writes on general-purpose buckets — pip, git, and sqlite all die there; goofys dormant since 2022), and every coherent one buys POSIX by reintroducing an always-on metadata service over a non-browsable chunk layout — the hub in another costume, minus the browsable bucket that is flint's differentiator. The closest cache shape, rclone's VFS write-back, has zero cross-client arbitration; its own docs call concurrent mounts a corruption risk. Nothing off the shelf provides browsable whole-file objects + an evicting working set + CAS-lease-guarded publishes + manifest/tombstone restore — and flint owns roughly two-thirds of that middle today. **flint-fuse is a third front end, not §7 brought forward:** the libflint plan's mount-pod tier is a libflint consumer serving exactly the lock/mmap/multi-writer contract flint-fuse excludes. Build one FUSE daemon with a pluggable backend — `tier::` over S3 now, libflint coordinates later — and amend §7 to name both, so the two FUSE deliverables converge instead of drifting. Treat FUSE passthrough (Linux 6.9, CAP_SYS_ADMIN, and it bypasses capture) and FUSE-over-io-uring (6.14, no fuser-crate support yet) as opportunistic accelerators, never floors — managed-node kernels in 2026 run 6.1–6.12, and the hub must remain the zero-consumer-requirements path. **Keep the placement dial in view:** the dial now starts a notch earlier — the lean checkout/publish sidecar (docs/flint-lean-architecture) needs no FUSE at all and is the direct-mode entry point; per-POD FUSE is the escalation for volumes too large or too lazy to materialize; per-NODE mount pods recover the shared cache, disk, and lease that per-POD sidecars multiply — the largest single lever on every duplication term on page 4 — at the cost of pod-level isolation. The hub stays. The ladder, for direct-mode shares, simply stops needing to exist.